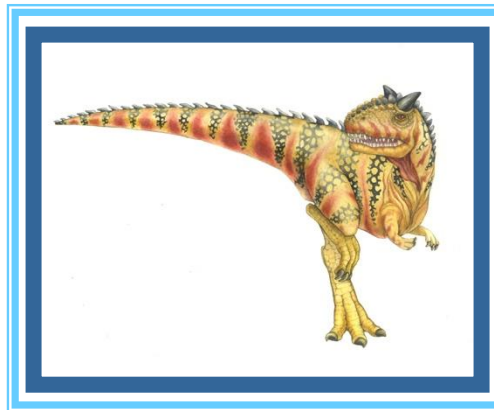


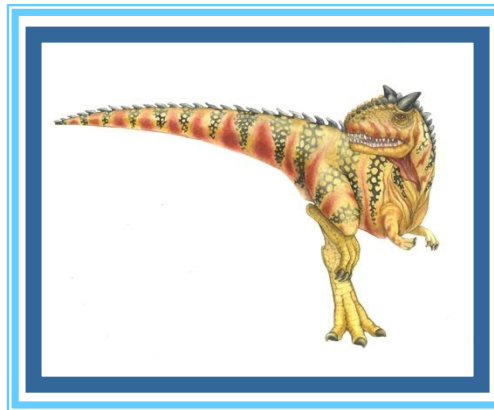
# These slides have been modified from the original



A huge thanks to the  
authors of Operating  
System Concepts for  
providing this resource



# These slides have been modified from the original



An indicator on a slide means I've modified something on the slide from the original slides

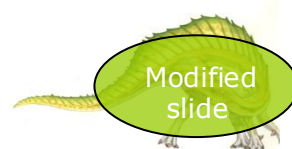
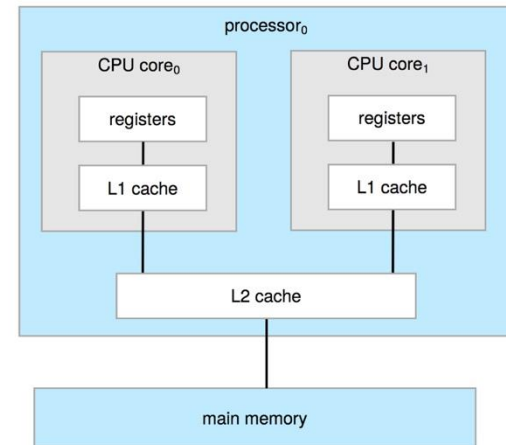
Modified slide

# Main Memory



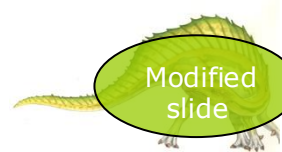
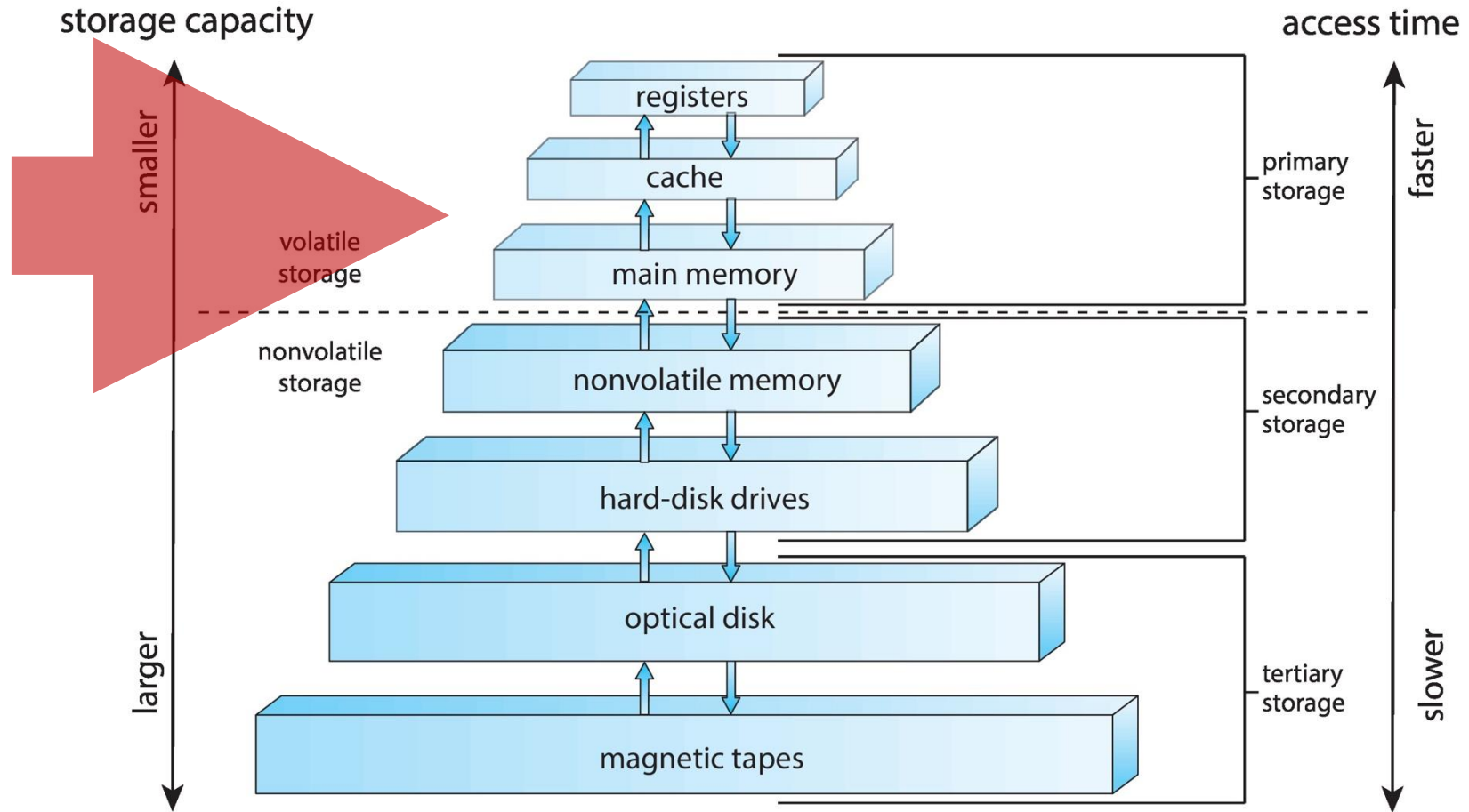
# Background

- Processes & operating system share memory
- And... Program must be brought (from disk) into memory and placed within a process for it to be run
- Main memory and registers are the only storage CPU can access directly
- So, memory unit only sees a stream of:
  - addresses & read requests, or
  - address & data and write requests
- Register access is done in one CPU clock (or less)
- Main memory can take many cycles, causing a **stall**
- **Cache** sits between main memory and CPU registers
- **Protection** of memory required to ensure correct operation





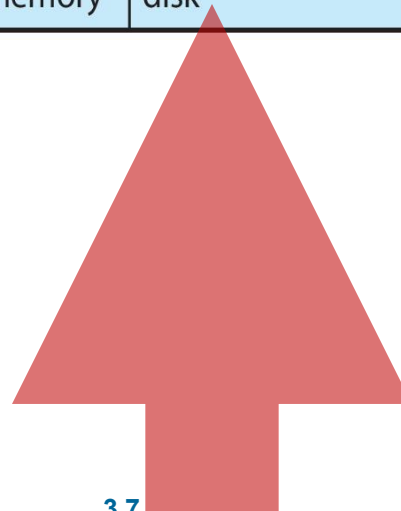
# Storage-Device Hierarchy





# Characteristics of Various Types of Storage

| Level                     | 1                                      | 2                             | 3                | 4                | 5                |
|---------------------------|----------------------------------------|-------------------------------|------------------|------------------|------------------|
| Name                      | registers                              | cache                         | main memory      | solid-state disk | magnetic disk    |
| Typical size              | < 1 KB                                 | < 16MB                        | < 64GB           | < 1 TB           | < 10 TB          |
| Implementation technology | custom memory with multiple ports CMOS | on-chip or off-chip CMOS SRAM | CMOS SRAM        | flash memory     | magnetic disk    |
| Access time (ns)          | 0.25-0.5                               | 0.5-25                        | 80-250           | 25,000-50,000    | 5,000,000        |
| Bandwidth (MB/sec)        | 20,000-100,000                         | 5,000-10,000                  | 1,000-5,000      | 500              | 20-150           |
| Managed by                | compiler                               | hardware                      | operating system | operating system | operating system |
| Backed by                 | cache                                  | main memory                   | disk             | disk             | disk or tape     |

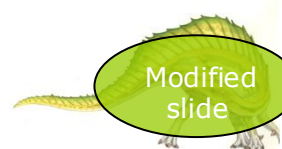
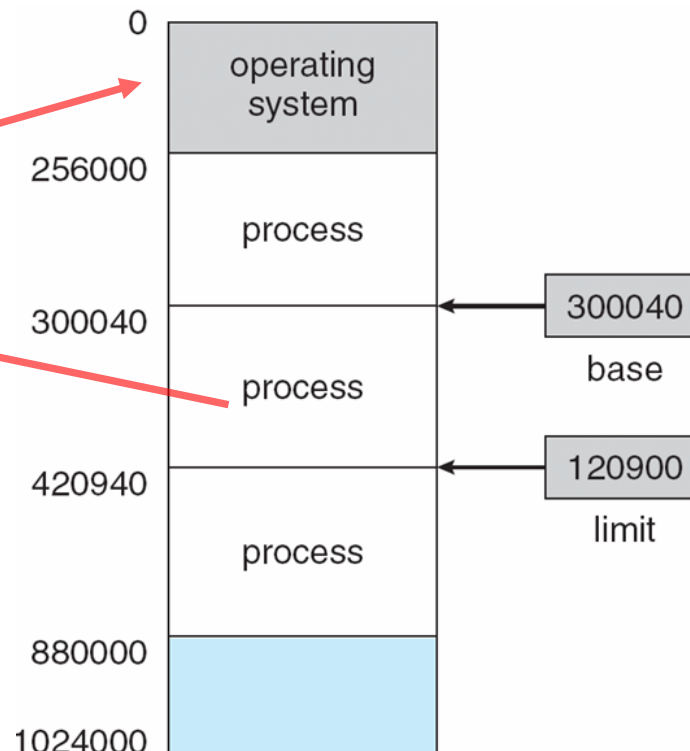




# Protection

- Need to ensure that a process can only access those addresses in its address space.
- We can provide this protection by using a pair of **base** and **limit registers** define the logical address space of a process

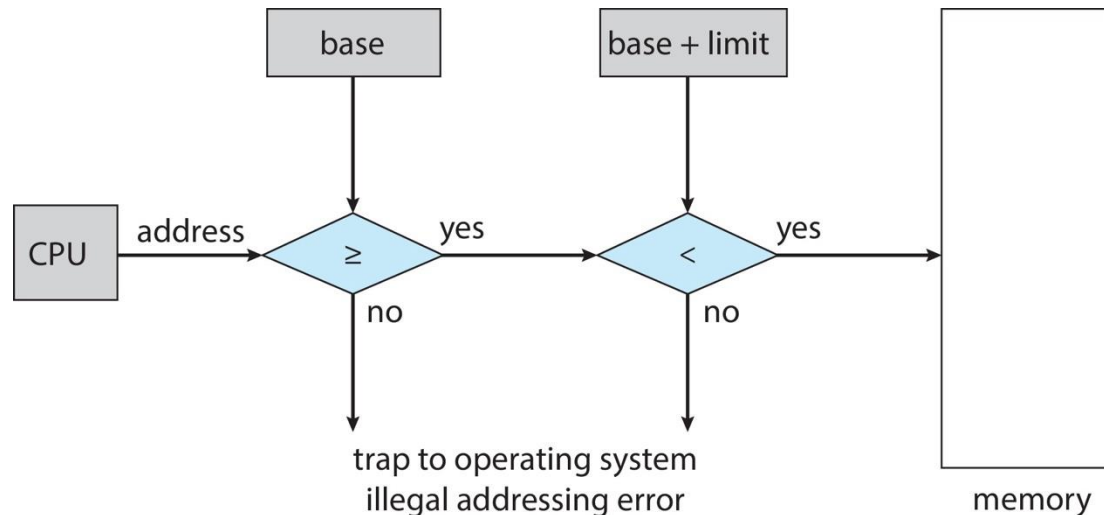
We don't want  
a process to be  
able to access  
memory where  
kernel resides!





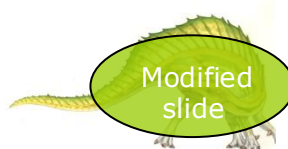
# Hardware Address Protection

- CPU must check every memory access generated in **user mode** to be sure it is between base and limit for that user



- instructions to load the **base and limit registers** are privileged (happen in **kernel mode**)

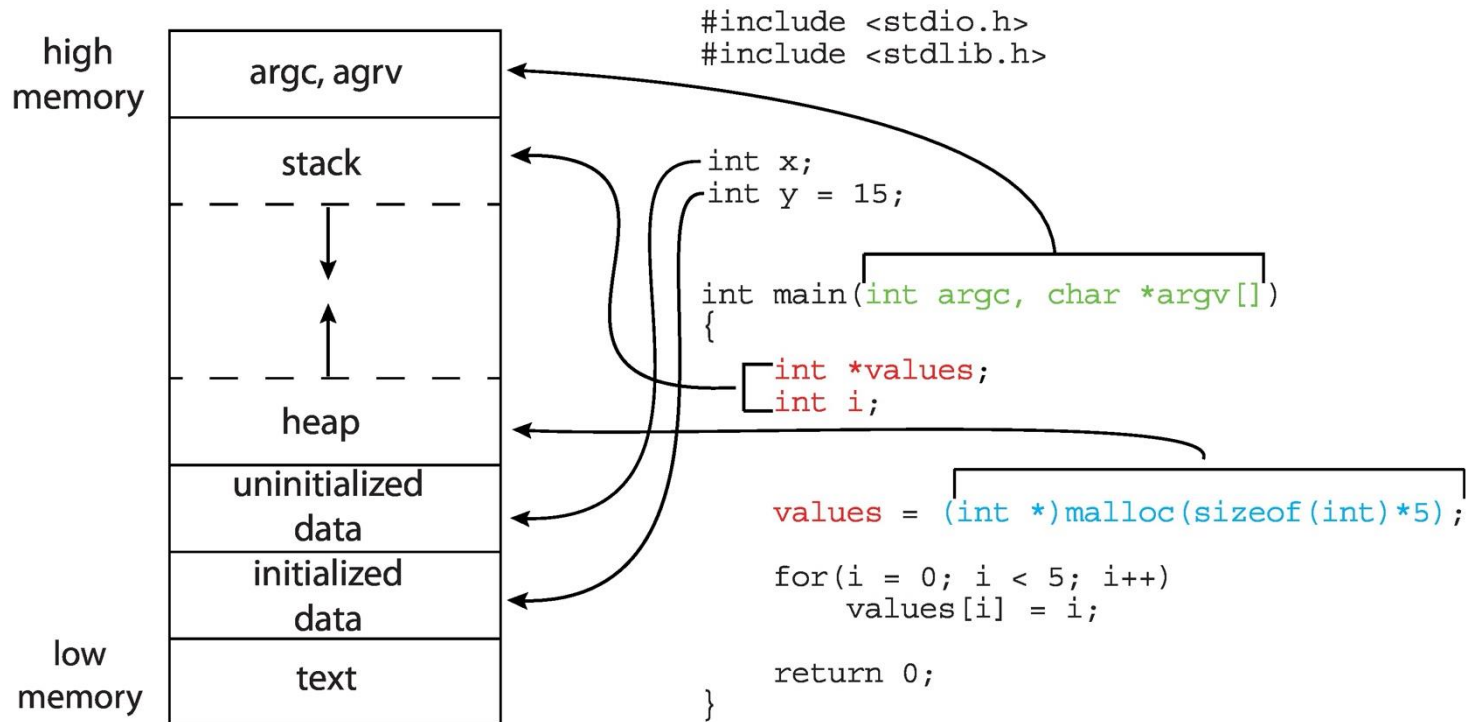
Also: There are two modes for our CPU, **user mode** and **kernel mode**.







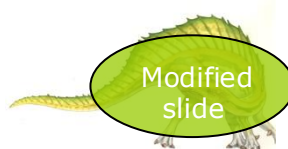
# Memory Layout of a C Program





# Address Binding

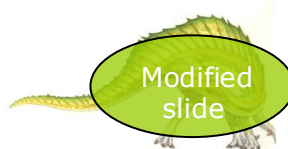
- Programs on disk, ready to be brought into memory to execute form an **input queue**
- **Addresses represented in different ways at different stages of a program's life**
  - Source code addresses usually symbolic
  - Compiled code addresses **bind** to **relocatable** addresses
    - ▶ i.e., “14 bytes from beginning of this module”
  - Linker or/ loader will bind relocatable addresses to absolute addresses
    - ▶ i.e., 74014
  - Each binding step maps one address space to another





# Binding of Instructions and Data to Memory

- Address binding of instructions and data to memory addresses can happen at three different stages
  - **Compile time:** If memory location known a priori, **absolute code** can be generated; must recompile code if starting location changes
  - **Load time:** Must generate **relocatable code** if memory location is not known at compile time
  - **Execution time:** **Binding** delayed until run time if the process can be moved during its execution from one memory segment to another
    - ▶ Need/Use hardware support for address maps (e.g., base and limit registers)

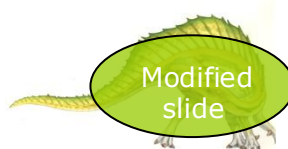




# Logical vs. Physical Address Space

---

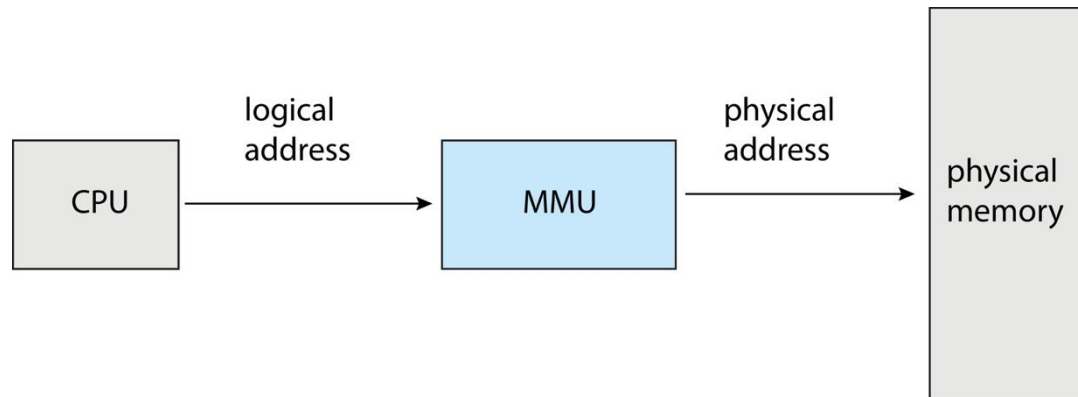
- The concept of a logical address space that is bound to a separate **physical address space** is central to proper memory management
  - **Logical address** – generated by the CPU; also referred to as **virtual address** (as seen by application)
  - **Physical address** – address seen by the memory unit
  
- Logical (virtual) and physical addresses differ in execution-time address-binding scheme
- **Logical address space** is the set of all logical addresses generated by a program
- **Physical address space** is the set of all physical addresses generated by a program



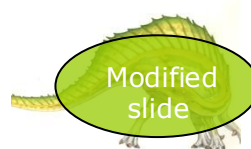
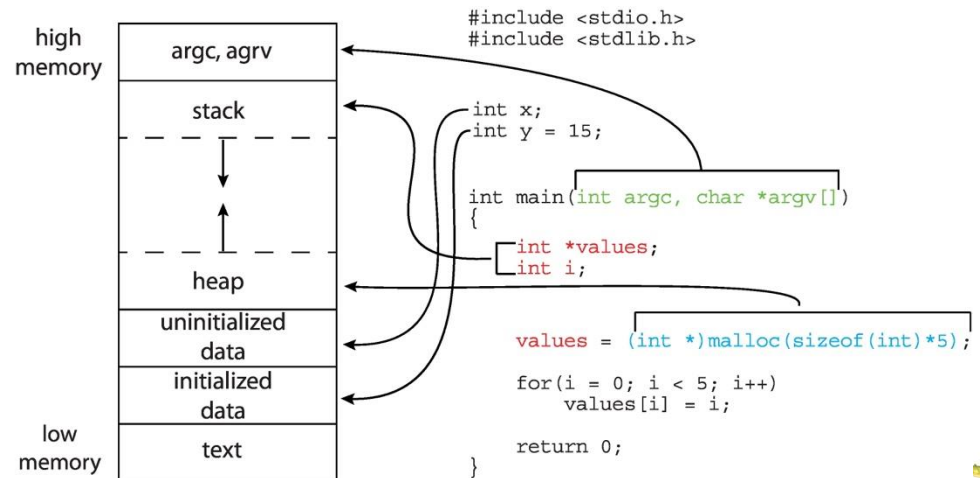


# Memory-Management Unit (MMU)

- There's a **Hardware device (MMU)** that at run time maps virtual (logical) to physical address



Many methods possible....

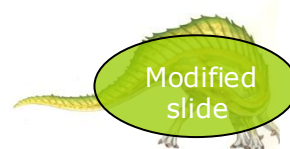
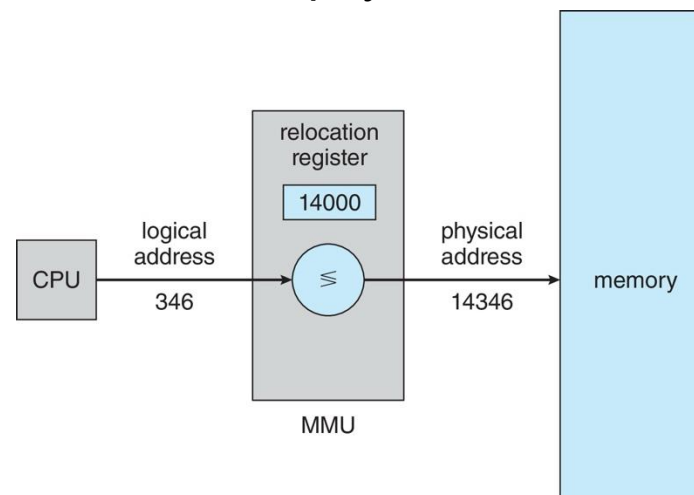


Modified slide



# Memory-Management Unit (Cont.)

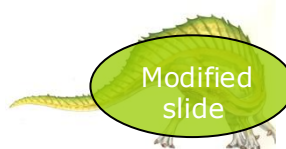
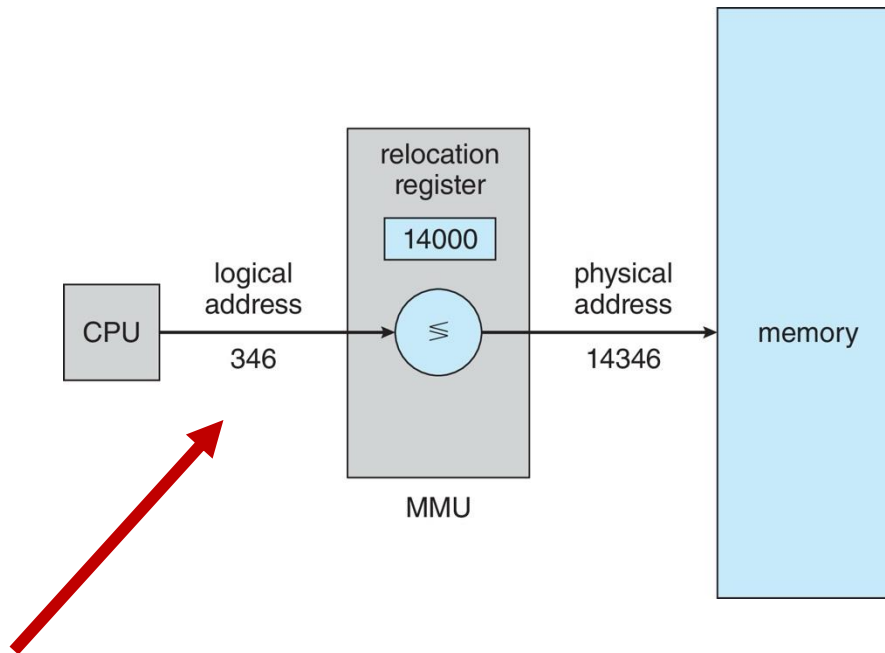
- Consider simple scheme which is a generalization of the **base-register** scheme.
- The base register now called **relocation register**
- The value in the relocation register is added to every address generated by a user process at the time it is sent to memory
- The user program deals with *logical* addresses; it never sees the *real* physical addresses
  - Execution-time binding occurs when reference is made to location in memory
  - Logical address bound to physical addresses





# Memory-Management Unit (Cont.)

- Consider simple scheme which is a generalization of the **base-register** scheme.
- The base register now called **relocation register**
- The value in the relocation register is added to every address generated by a user process at the time it is sent to memory

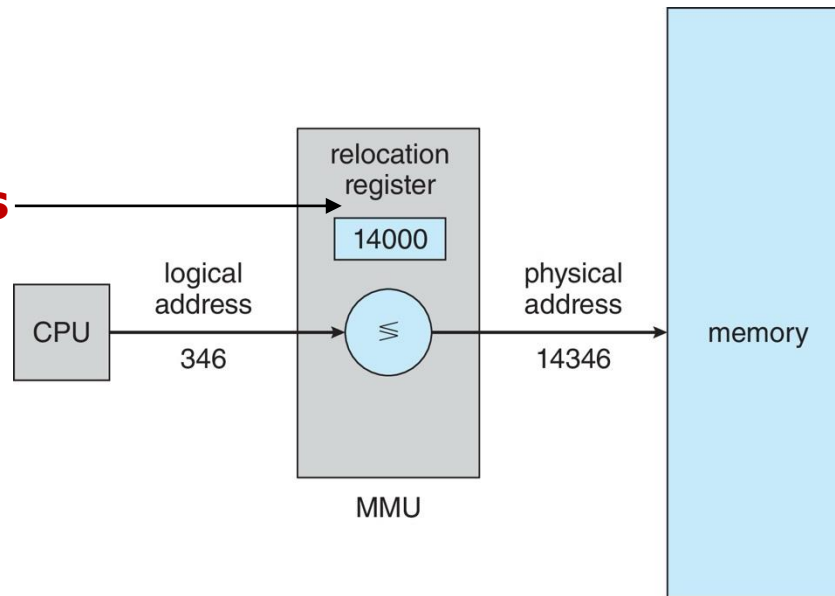




# Memory-Management Unit (Cont.)

- Consider simple scheme which is a generalization of the **base-register** scheme.
- The base register now called **relocation register**
- The value in the relocation register is added to every address generated by a user process at the time it is sent to memory

**How would this get set? By the scheduling dispatcher, while in kernel mode**



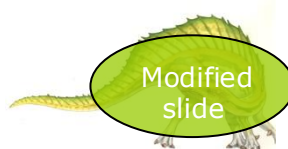




# Contiguous Allocation

---

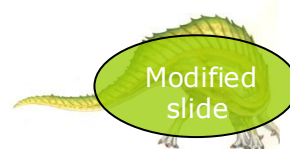
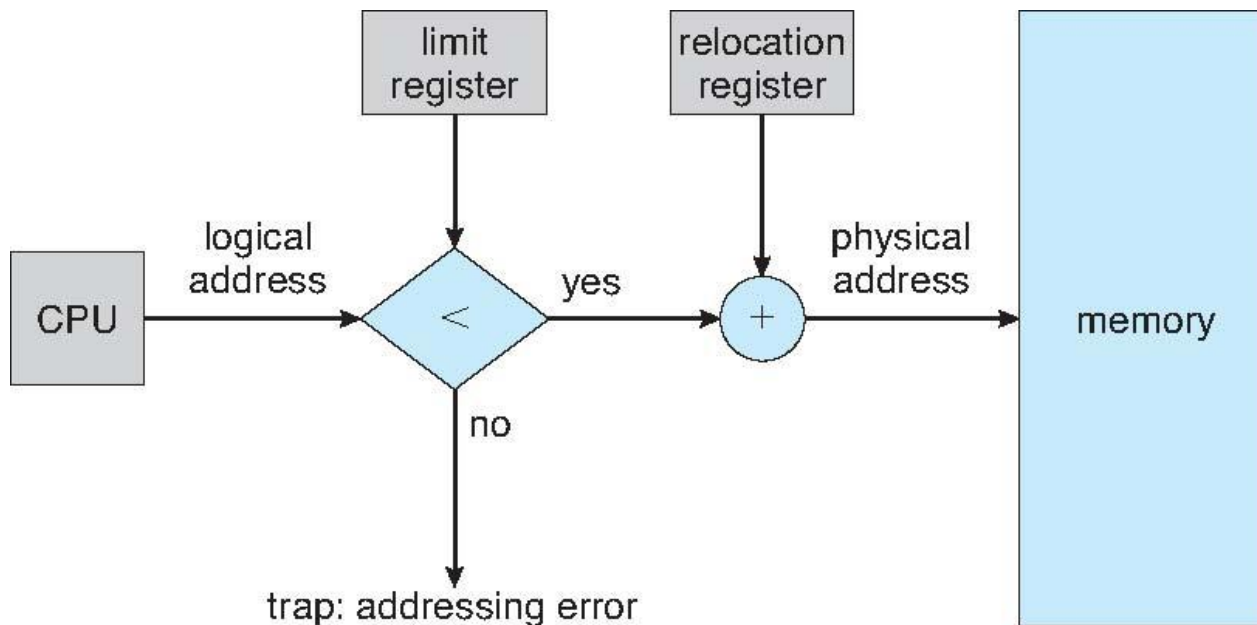
- Main memory must support both OS and user processes
- Limited resource, must allocate efficiently
- Contiguous allocation is **one early method**
- Main memory usually into two **partitions**:
  - Resident operating system, usually held in low memory with **interrupt vector table**
  - User processes then held in high memory
  - Each process contained in single contiguous section of memory





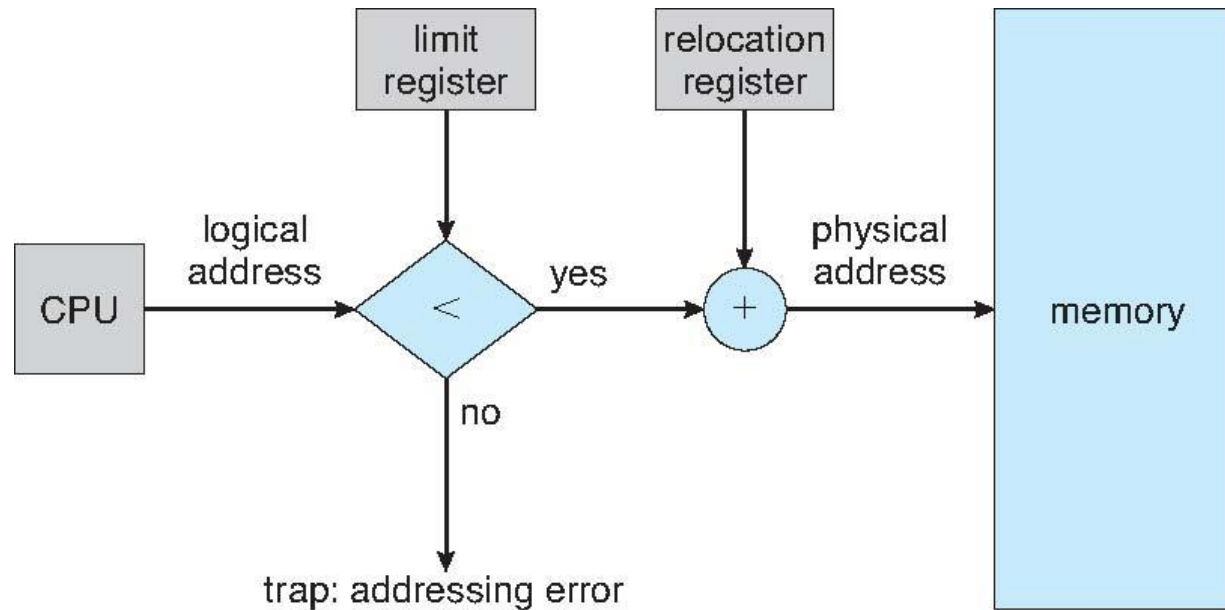
# Contiguous Allocation (Cont.)

- Can use Relocation registers to **protect** user processes from each other, and from changing operating-system code and data
  - Base register contains value of smallest physical address
  - Limit register contains range of logical addresses – each logical address must be less than the limit register
  - MMU maps logical address *dynamically*





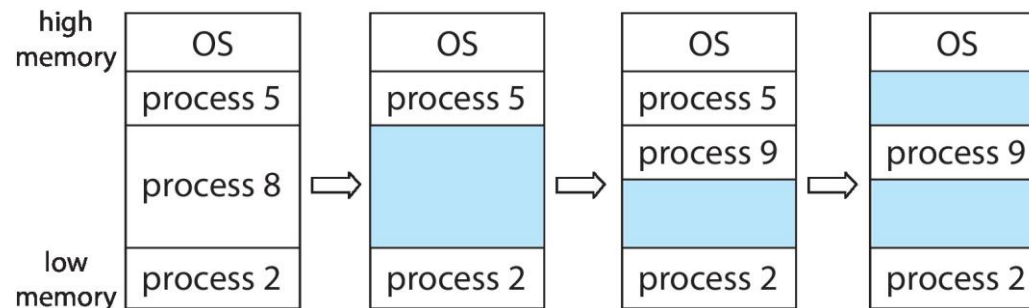
# Hardware Support for Relocation and Limit Registers





# Variable Partition Scheme

- Multiple-partition allocation
  - Degree of multiprogramming limited by number of partitions
  - **Variable-partition** sizes for efficiency (sized to a given process' needs)
  - **Hole** – block of available memory; holes of various size are scattered throughout memory
  - When a process arrives, it is allocated memory from a hole large enough to accommodate it
  - Process exiting frees its partition, adjacent free partitions combined
  - Operating system maintains information about:
    - a) allocated partitions    b) free partitions (hole)





# Dynamic Storage-Allocation Problem

---

How to satisfy a request of size  $n$  from a list of free holes?

- **First-fit**: Allocate the **first** hole that is big enough
- **Best-fit**: Allocate the **smallest** hole that is big enough; must search entire list, unless ordered by size
  - Produces the smallest leftover hole
- **Worst-fit**: Allocate the **largest** hole; must also search entire list
  - Produces the largest leftover hole

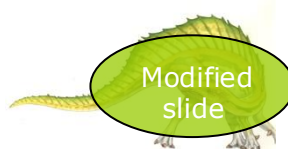
First-fit and best-fit better than worst-fit in terms of speed and storage utilization





# Fragmentation

- **External Fragmentation** – total memory space **may** exist to satisfy a request, but it is not contiguous (**too many little pieces here and there**)
  - Approaches like **compaction** – shuffle the the memory contents to put the free memory next to each other.
  - ...
- **Internal Fragmentation** – allocated memory may be slightly larger than requested memory; this size difference is memory internal to a partition, but not being used (this is the case where we break memory into blocks).



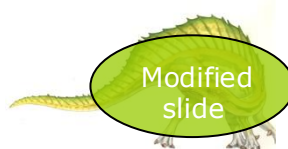
# Problems....

- These MM approaches seem problematic and wasteful... E.g. external fragmentation (waste!)
- There's a better way: Paging



# Paging

- The idea ... Physical address space of a process can be noncontiguous; process is allocated physical memory where available
  - Avoids external fragmentation
  - Avoids problem of varying sized memory chunks
- Divide physical memory into fixed-sized blocks called **frames**
  - Size is power of 2, between 512 bytes and 16 Mbytes
- Divide logical memory into blocks of same size called **pages**
- Keep track of all free frames
- To run a program of size ***N*** pages, need to find ***N*** free frames and load program
- Set up a **page table** to translate logical to physical addresses
- **Backing store** likewise split into pages (more later)
- Can (probably will) still have Internal fragmentation at %

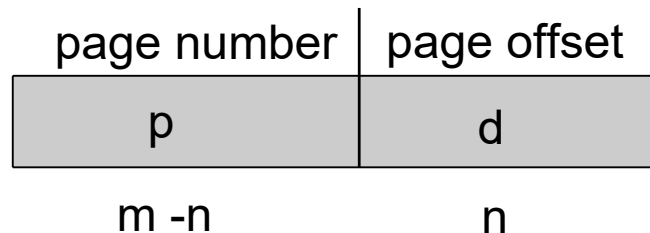




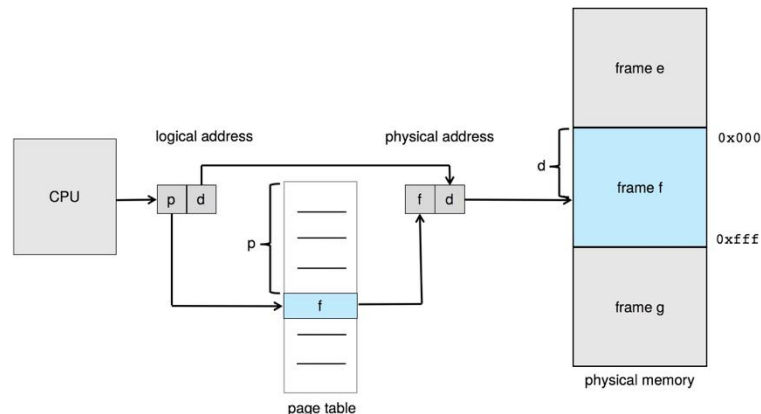


# Address Translation Scheme

- Address generated by CPU is divided into:
  - **Page number** ( $p$ ) – used as an index into a **page table** which contains base address of each page in physical memory
  - **Page offset** ( $d$ ) – combined with base address to define the physical memory address that is sent to the memory unit



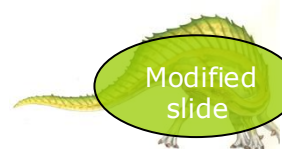
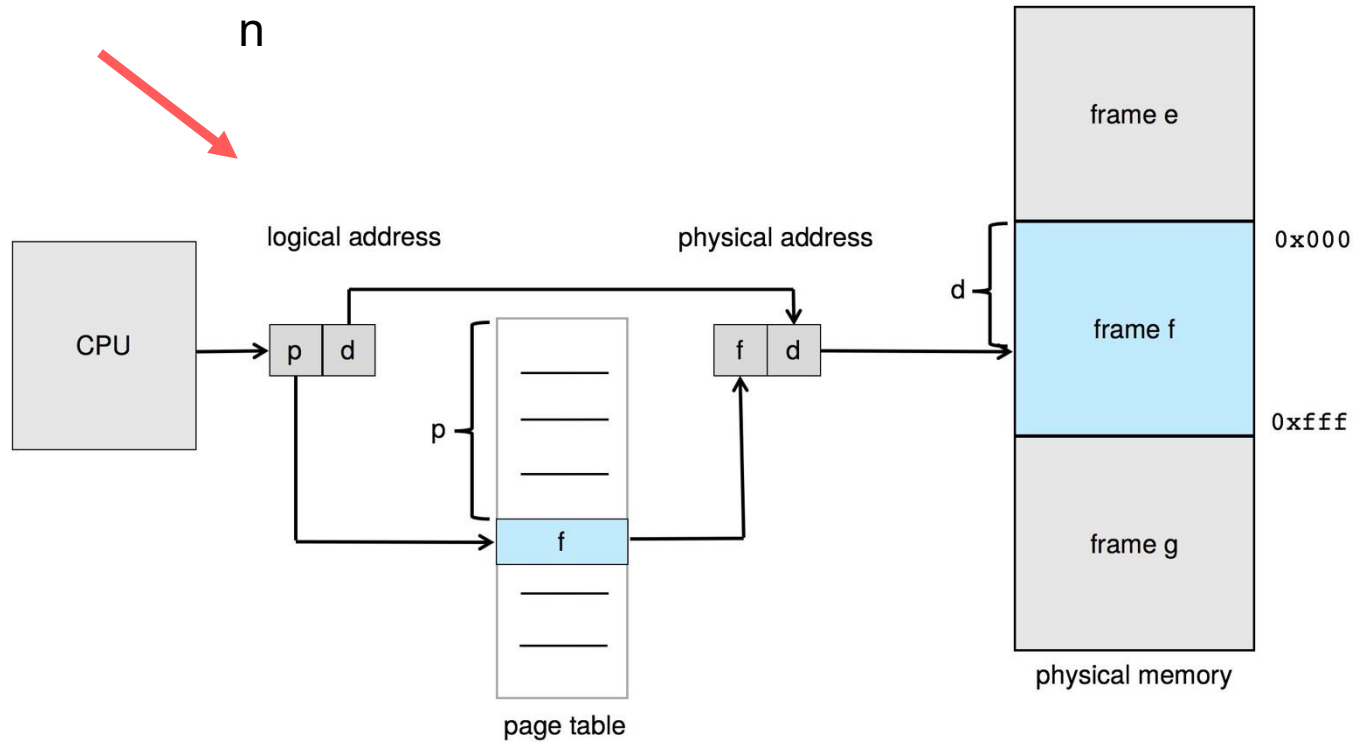
- For given logical address space  $2^m$  and page size  $2^n$





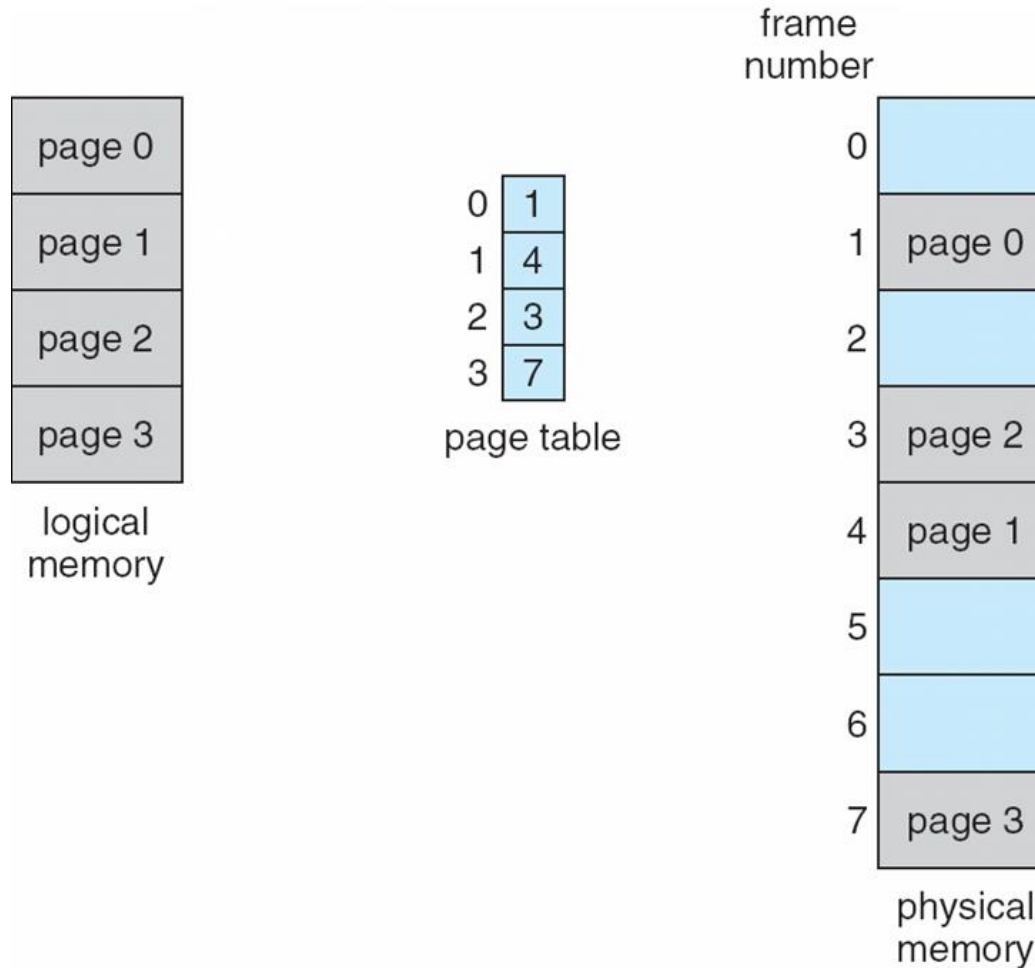
# Paging Hardware

| page number | page offset |
|-------------|-------------|
| p           | d           |
| m - n       | n           |





# Paging Model of Logical and Physical Memory





# Paging Example

- Logical address:  $n = 2$  and  $m = 4$ . Using a page size of 4 bytes and a physical memory of 32 bytes (8 pages)

|    |   |
|----|---|
| 0  | a |
| 1  | b |
| 2  | c |
| 3  | d |
| 4  | e |
| 5  | f |
| 6  | g |
| 7  | h |
| 8  | i |
| 9  | j |
| 10 | k |
| 11 | l |
| 12 | m |
| 13 | n |
| 14 | o |
| 15 | p |

logical memory

|   |   |
|---|---|
| 0 | 5 |
| 1 | 6 |
| 2 | 1 |
| 3 | 2 |

page table

|    |                  |
|----|------------------|
| 0  |                  |
| 4  | i<br>j<br>k<br>l |
| 8  | m<br>n<br>o<br>p |
| 12 |                  |
| 16 |                  |
| 20 | a<br>b<br>c<br>d |
| 24 | e<br>f<br>g<br>h |
| 28 |                  |

physical memory



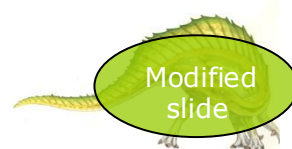


# Paging -- Calculating internal fragmentation

- We've no external fragmentation... But we still have **internal fragmentation. Why?**
  - Example... Page size = 2,048 bytes
  - Process size = 72,766 bytes
  - 35 pages + 1,086 bytes (will need to be allocated 36 frames/pages)
  - Internal fragmentation of  $2,048 - 1,086 = 962$  bytes
  - Worst case fragmentation = 1 frame – 1 byte (e.g.  $2048 - 1$ )
  - On average fragmentation =  $1 / 2$  frame size
  - .... so a small frame sizes is desirable? (what about **overhead?**)
  - Smaller it gets, larger our table!
    - And each page table entry takes memory to track
- 
- Page sizes have grown over time
    - Solaris supports two page sizes – 8 KB and 4 MB

```
(base) ham@hammerGPU:~$ getconf PAGESIZE is 4096
```

But each page table entry takes



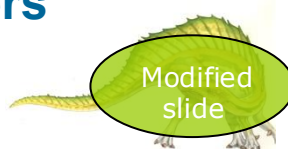


# Implementation of Page Table

---

Where is the page table?

- Page table is kept in main memory
  - For example, registers used (simple overview):
    - ▶ **Page-table base register (PTBR)** points to the page table
    - ▶ **Page-table length register (PTLR)** indicates size of the page table
- **Note... only using this scheme means every data/instruction access would require two memory accesses**
  - One for the page table and one for the data / instruction
- The **two-memory access problem** can be solved by the use of a special fast-lookup hardware cache called **translation look-aside buffers (TLBs)** (also called **associative memory**).



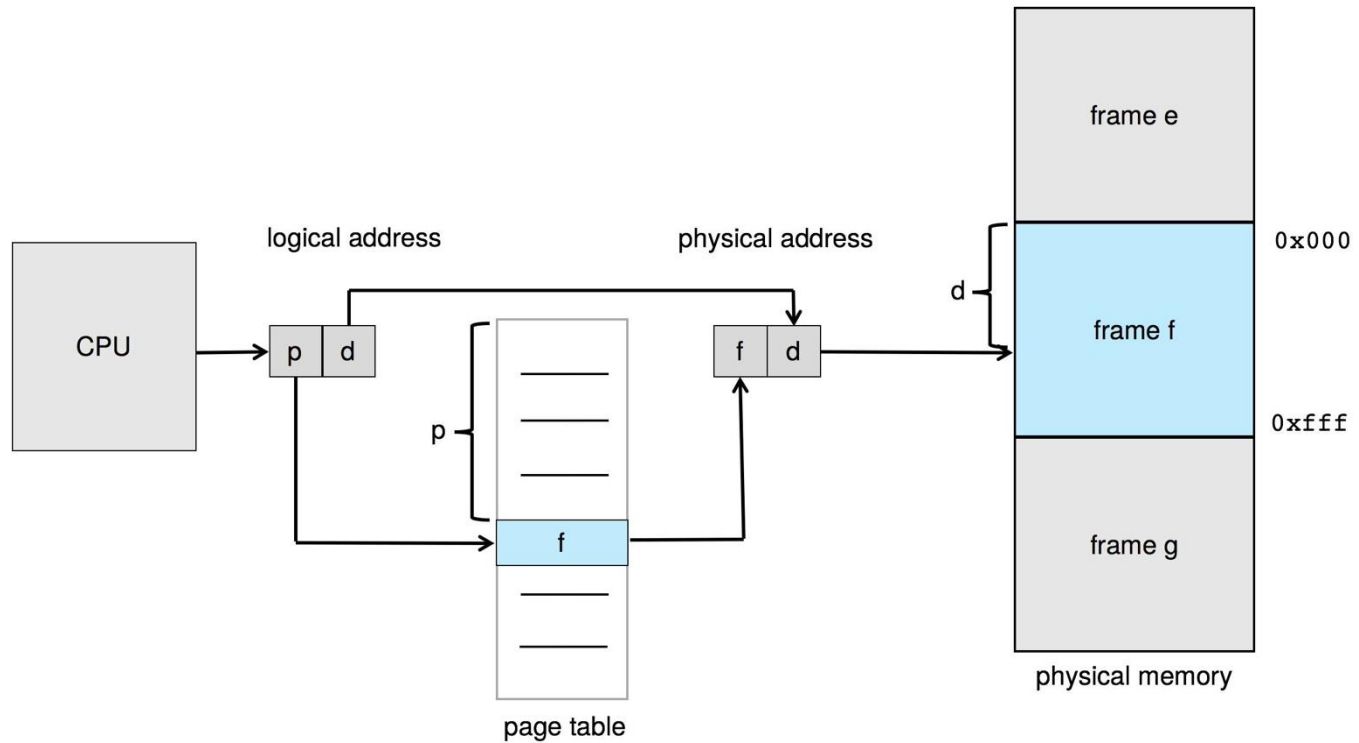


| Level                     | 1                                      | 2                             | 3                | 4                | 5                |
|---------------------------|----------------------------------------|-------------------------------|------------------|------------------|------------------|
| Name                      | registers                              | cache                         | main memory      | solid-state disk | magnetic disk    |
| Typical size              | < 1 KB                                 | < 16MB                        | < 64GB           | < 1 TB           | < 10 TB          |
| Implementation technology | custom memory with multiple ports CMOS | on-chip or off-chip CMOS SRAM | CMOS SRAM        | flash memory     | magnetic disk    |
| Access time (ns)          | 0.25-0.5                               | 0.5-25                        | 80-250           | 25,000-50,000    | 5,000,000        |
| Bandwidth (MB/sec)        | 20,000-100,000                         | 5,000-10,000                  | 1,000-5,000      | 500              | 20-150           |
| Managed by                | compiler                               | hardware                      | operating system | operating system | operating system |
| Backed by                 | cache                                  | main memory                   | disk             | disk             | disk or tape     |





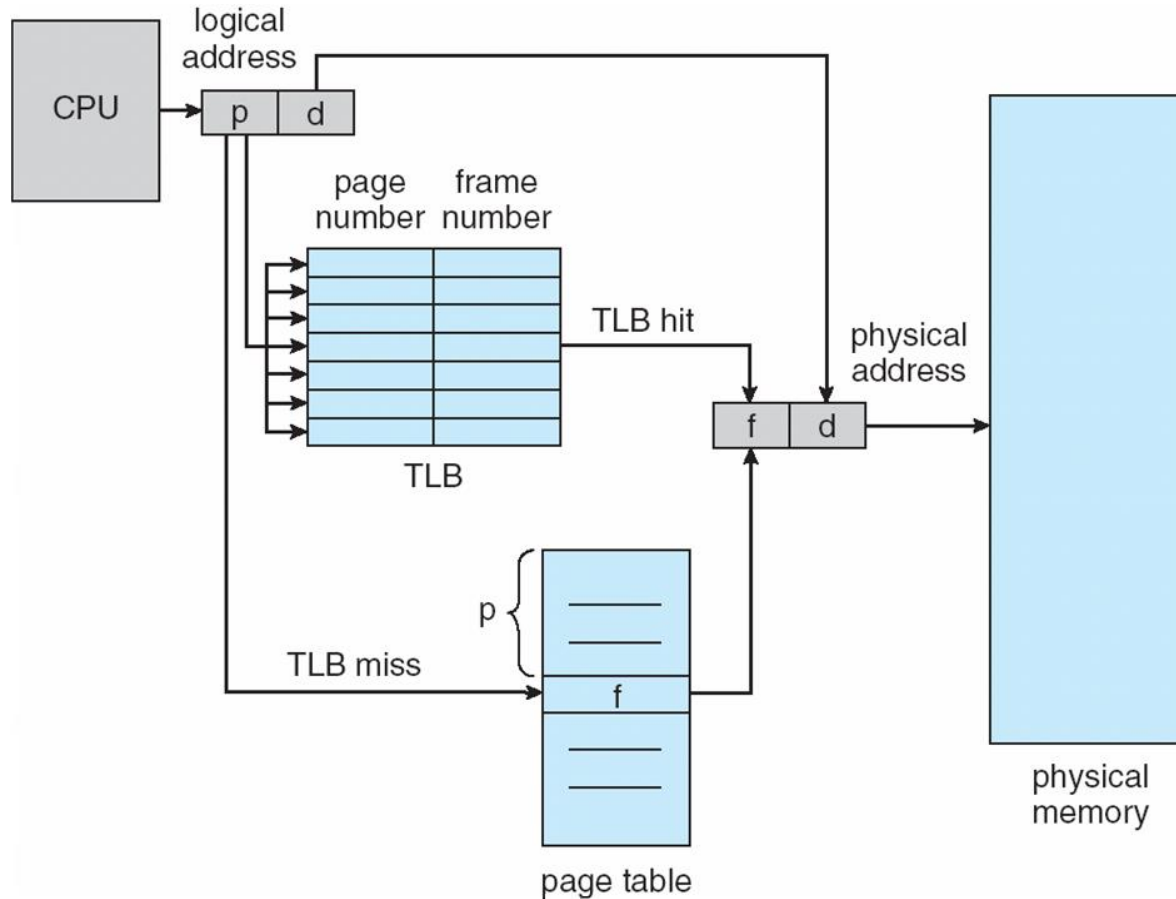
# Paging Hardware (no TLB)







# Paging Hardware With TLB



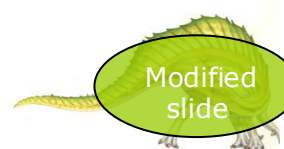
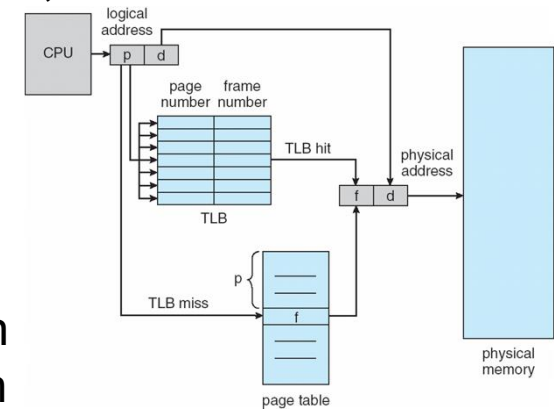


# Translation Look-Aside Buffer

| Page # | Frame # |
|--------|---------|
|        |         |
|        |         |
|        |         |
|        |         |

TLBs typically small (64 to 1,024 entries)

- On a **TLB cache miss**, value is loaded into the TLB for faster access next time
  - Replacement policies must be considered (LRU, etc)
  - Some entries can be **“wired down”** for permanent fast access
- TLBs store **address-space identifiers (ASIDs)** in each TLB entry – these can uniquely identify each process to provide address-space protection for that process (talk about this in a while)





# Hardware

- Associative memory – parallel search

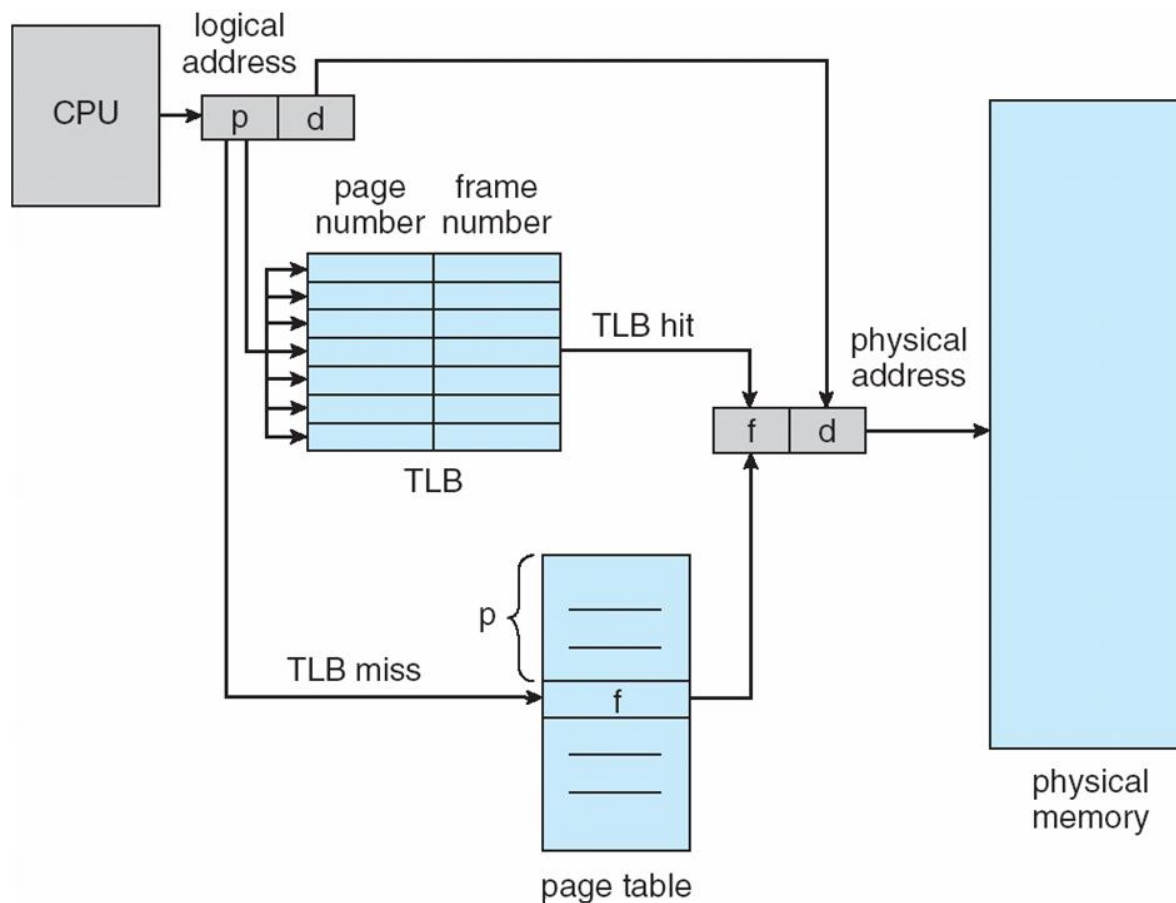
| Page # | Frame # |
|--------|---------|
|        |         |
|        |         |
|        |         |
|        |         |

- Address translation (p, d)
  - If p is in associative register, get frame # out
  - Otherwise get frame # from page table in memory





# Paging Hardware With TLB



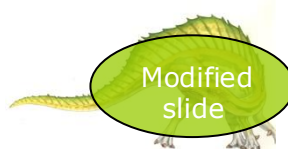


# Effective Access Time

- **Hit ratio** – percentage of times that a page number is found in the **TLB**
- An 80% hit ratio means that we find the desired page number in the TLB 80% of the time.
- Suppose it takes 10 nanoseconds to access memory.
  - If we find the desired page in TLB then a mapped-memory access take 10 ns (*disregard the small amount of time to access cache*)
  - Otherwise, we need two memory accesses so it is 20 ns
- **Effective Access Time (EAT)** (average)
$$\text{EAT} = 0.80 \times 10 + 0.20 \times 20 = 12 \text{ nanoseconds}$$

*implying 20% slowdown in access time*
- Consider a more realistic hit ratio of 99%,
$$\text{EAT} = 0.99 \times 10 + 0.01 \times 20 = 10.1\text{ns}$$

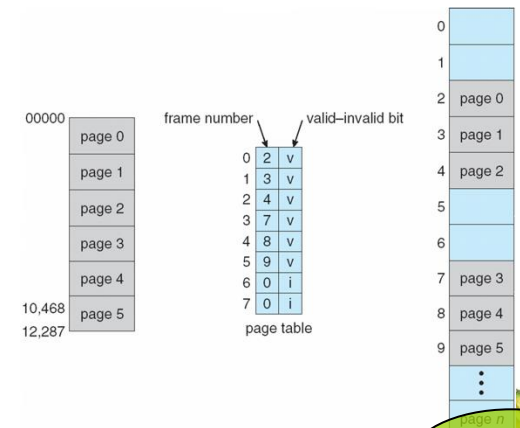
*implying only 1% slowdown in access time.*



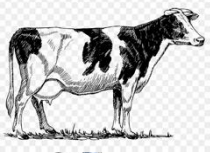


# Memory Protection (page tables)

- **Other features:** Memory protection implemented by associating protection bit with each frame to indicate if **read-only** or **read-write** and/or **execute** access is allowed
- **Valid-invalid** bit attached to each entry in the page table:
  - “**valid**” indicates that the associated page is in the process’ logical address space, and is thus a legal page
  - “**invalid**” indicates that the page is not in the process’ logical address space / or not in memory
- Any violations result in a trap to the kernel



Modified slide



# Valid (v) or Invalid (i) Bit In A Page Table

|        |        |
|--------|--------|
| 00000  | page 0 |
|        | page 1 |
|        | page 2 |
|        | page 3 |
|        | page 4 |
| 10,468 | page 5 |
| 12,287 |        |

| frame number |   | valid-invalid bit |
|--------------|---|-------------------|
| 0            | 2 | v                 |
| 1            | 3 | v                 |
| 2            | 4 | v                 |
| 3            | 7 | v                 |
| 4            | 8 | v                 |
| 5            | 9 | v                 |
| 6            | 0 | i                 |
| 7            | 0 | i                 |

page table

|   |               |
|---|---------------|
| 0 |               |
| 1 |               |
| 2 | page 0        |
| 3 | page 1        |
| 4 | page 2        |
| 5 |               |
| 6 |               |
| 7 | page 3        |
| 8 | page 4        |
| 9 | page 5        |
|   | ⋮             |
|   | page <i>n</i> |





# Valid-Invalid Bit

- With each page table entry a valid–invalid bit is associated (**v**  $\Rightarrow$  in-memory – **memory resident**, **i**  $\Rightarrow$  not-in-memory)
- Initially valid–invalid bit is set to **i** on all entries
- Example of a page table snapshot:

| Frame # | valid-invalid bit |
|---------|-------------------|
|         |                   |
|         | <b>v</b>          |
|         | <b>v</b>          |
|         | <b>v</b>          |
|         | <b>i</b>          |
| ...     |                   |
|         | <b>i</b>          |
|         | <b>i</b>          |

page table

- During MMU address translation, if valid–invalid bit in page table entry is **i**  $\Rightarrow$  page fault







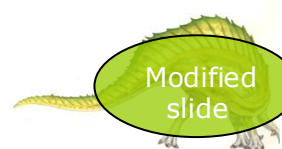
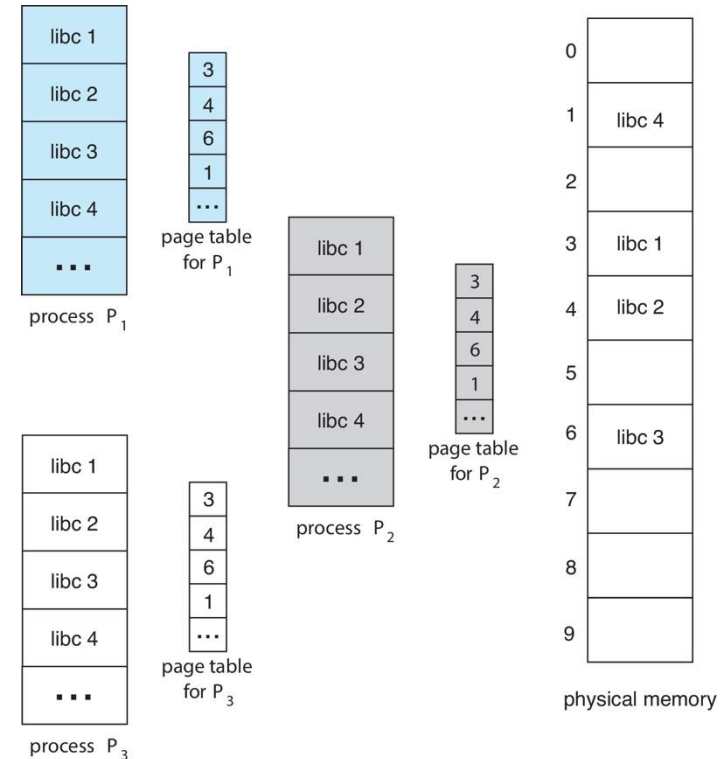
# Shared Pages

## ■ Shared code (advantage)

- One copy of read-only code shared among processes (i.e., text editors, compilers, window systems)
- Similar to multiple threads sharing the same process space
- Also useful for interprocess communication if sharing of read-write pages is allowed

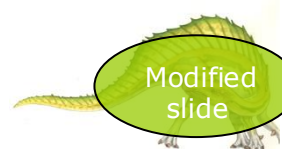
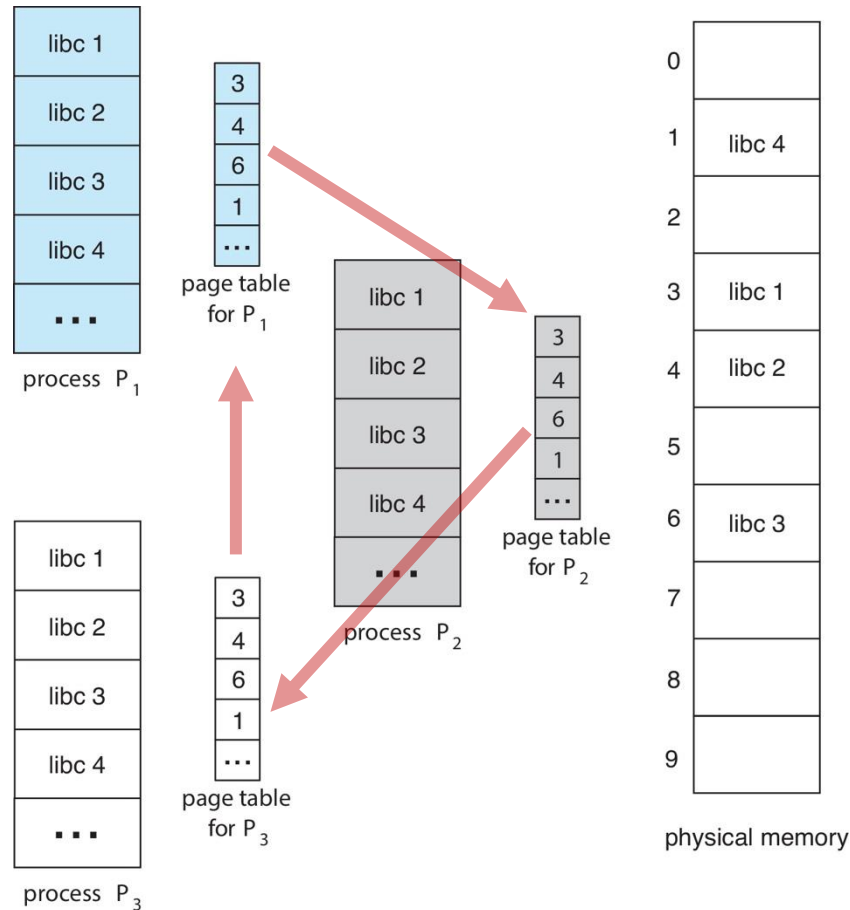
## ■ Private code and data

- Each process keeps a separate copy of the code and data





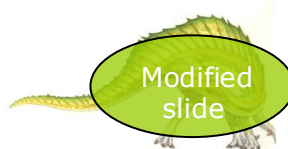
# Shared Pages Example





# Structure of the Page Table

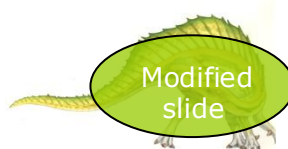
- Memory structures for paging can get large using straight-forward methods
  - Consider a 32-bit logical address space as on modern computers
  - Page size of 4 KB ( $2^{12}$ )
  - Page table would have 1 million entries ( $2^{20} / 2^{12}$ )
  - If each entry is 4 bytes → each process 4 MB of physical address space for the page table alone
    - ▶ 4mb is okay, but if larger, perhaps we don't want to allocate that **contiguously** in main memory
- What about 48-bit address, with 4 KB ( $2^{12}$ ) page size?
  - ( $2^{36}$ ) = 68719476736 \* 6 bytes (assuming) = 412316.860416 ... (412 GiB)
- We need to think about sensible data structures for page tables





# Structure of the Page Table

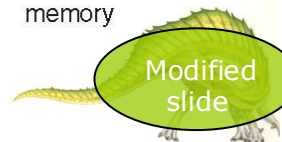
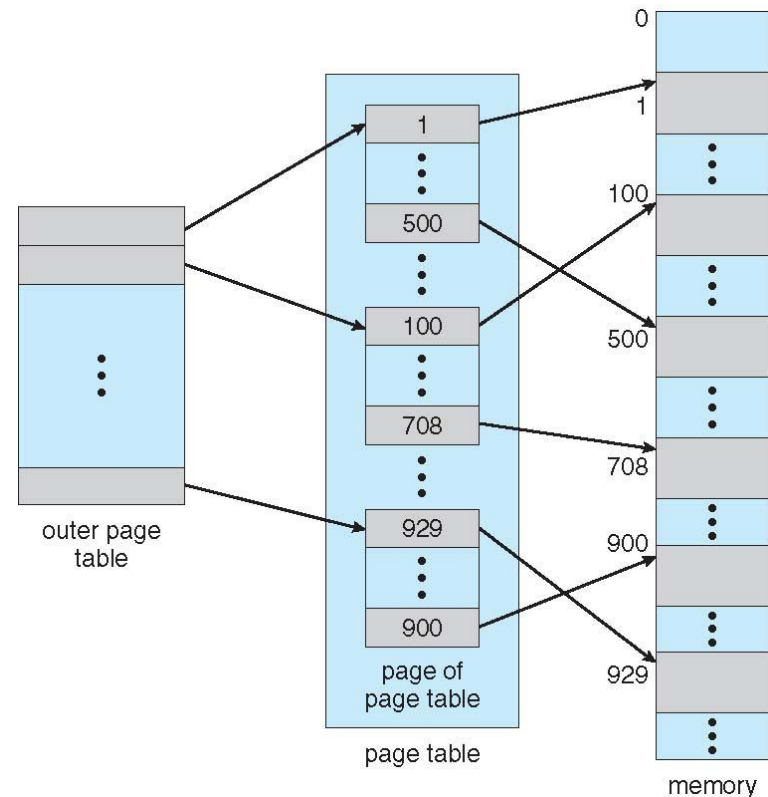
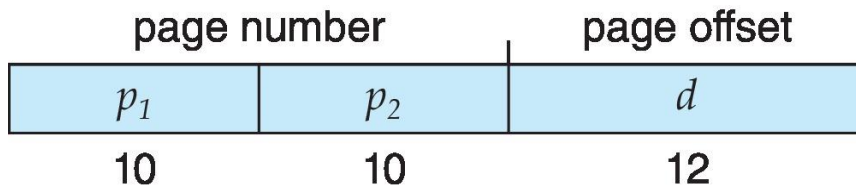
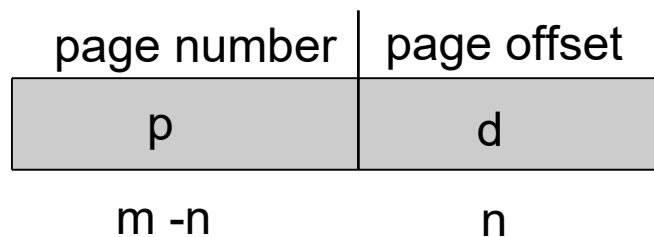
- Memory structures for paging can get large using straight-forward methods
  - Consider a 32-bit logical address space as on modern computers
  - Page size of 4 KB ( $2^{12}$ )
  - Page table would have 1 million entries ( $2^{20} / 2^{12}$ )
  - If each entry is 4 bytes → each process 4 MB of physical address space for the page table alone
    - ▶ Don't want to allocate that **contiguously** in main memory
- One simple solution is to divide the page table into smaller units, and there's a few different ways to do this:
  - ▶ Hierarchical Paging < - we can *scatter* parts of the page table across memory
  - ▶ Hashed Page Tables (other solution)
  - ▶ Inverted Page Tables (other solution)





# Hierarchical Page Tables

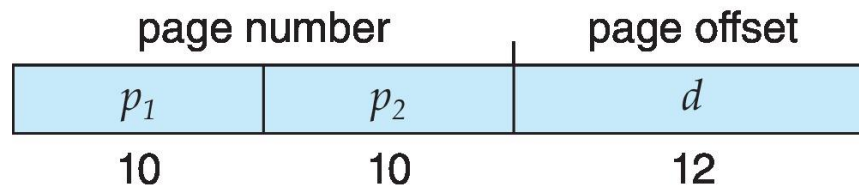
- Break up the logical address space into multiple page tables
- A simple technique is a two-level page table
- We then page the page table





# Two-Level Paging Example

- A logical address (on 32-bit machine with 4K page size) is divided into:
  - a page number consisting of 20 bits
  - a page offset consisting of 12 bits
- Since the page table is paged, the page number is further divided into:
  - a 10-bit page number
  - a 10-bit page offset
- Thus, a logical address is as follows:

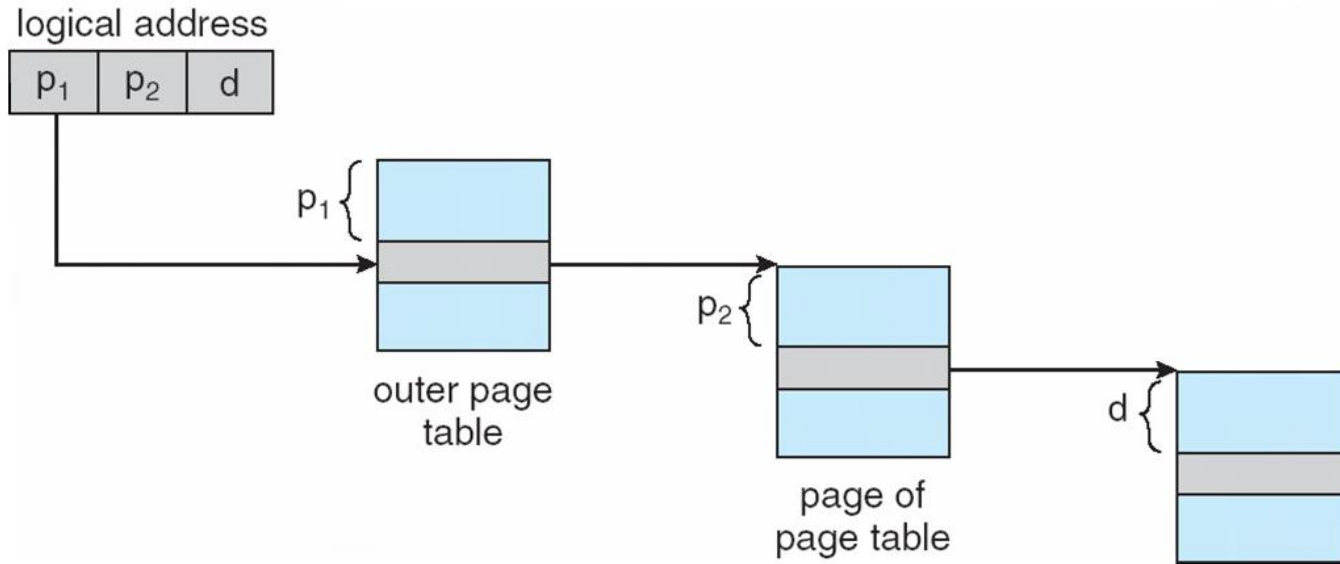


- where  $p_1$  is an index into the outer page table, and  $p_2$  is the displacement within the page of the inner page table
- Known as **forward-mapped page table**





# Address-Translation Scheme





# 64-bit Logical Address Space

- Even two-level paging scheme not sufficient
- If page size is 4 KB ( $2^{12}$ )
  - Then page table has  $2^{52}$  entries
  - If two level scheme, inner page tables could be  $2^{10}$  8-byte entries
  - Address would look like

| outer page | inner page | offset |
|------------|------------|--------|
| $p_1$      | $p_2$      | $d$    |
| 42         | 10         | 12     |

- Outer page table has  $2^{42}$  entries or  $2^{44}$  bytes
- One solution is to add a 2<sup>nd</sup> outer page table
- But in the following example the 2<sup>nd</sup> outer page table is still  $2^{34}$  bytes in size
  - ▶ And 4 memory access to get to one physical memory location

| 2nd outer page | outer page | inner page | offset |
|----------------|------------|------------|--------|
| $p_1$          | $p_2$      | $p_3$      | $d$    |
| 32             | 10         | 10         | 12     |

Modified  
slide

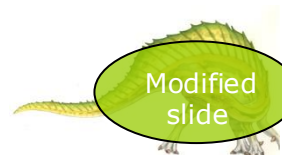




# Three-level Paging Scheme

| outer page | inner page | offset |
|------------|------------|--------|
| $p_1$      | $p_2$      | $d$    |
| 42         | 10         | 12     |

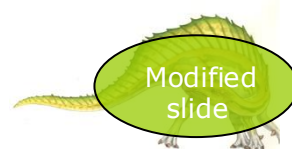
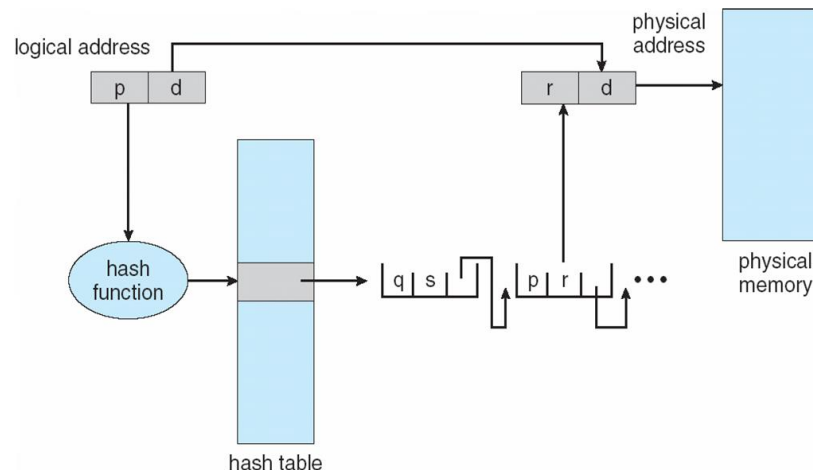
| 2nd outer page | outer page | inner page | offset |
|----------------|------------|------------|--------|
| $p_1$          | $p_2$      | $p_3$      | $d$    |
| 32             | 10         | 10         | 12     |





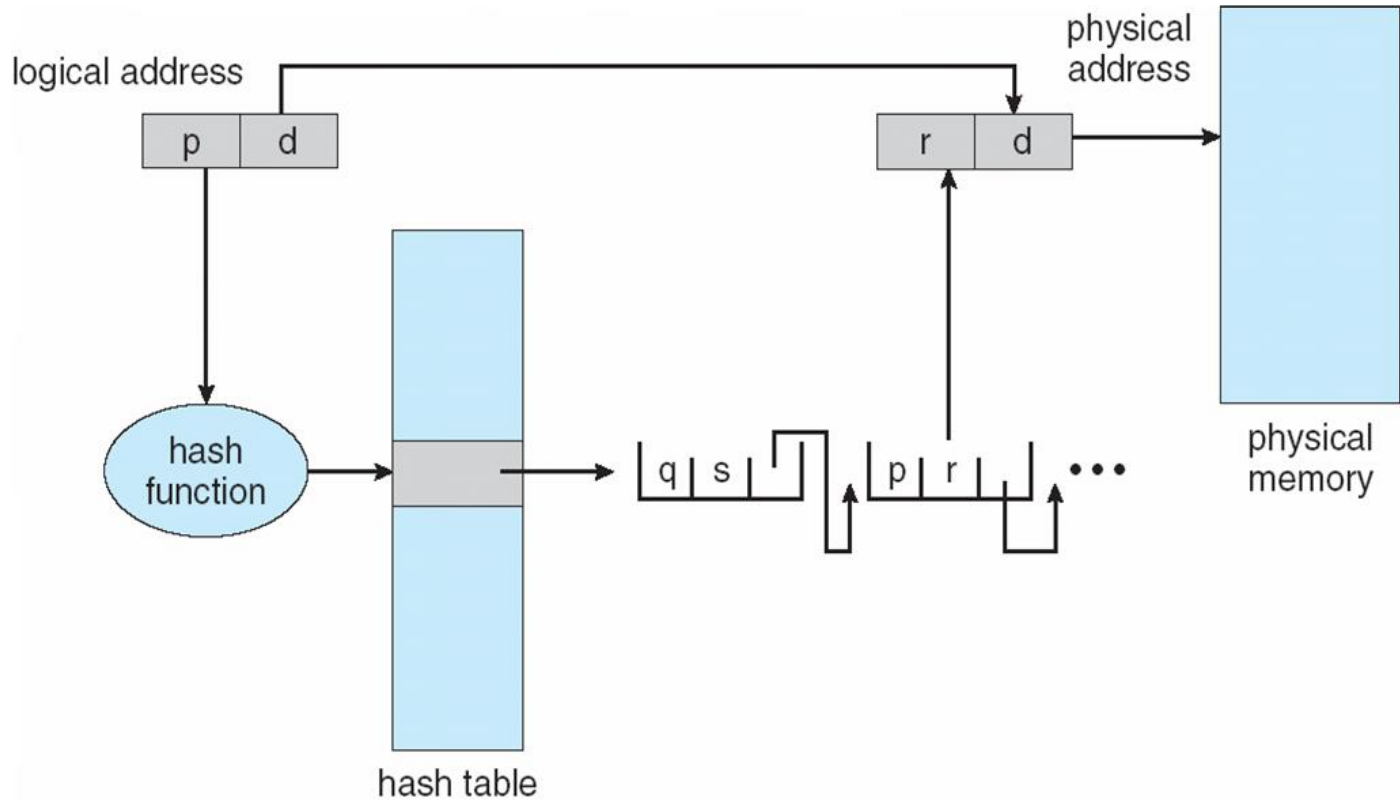
# Hashed Page Tables

- Common in address spaces  $> 32$  bits
- The virtual page number is hashed into a page table
  - This page table contains a chain of elements hashing to the same location
- Each element contains (1) the virtual page number (2) the value of the mapped page frame (3) a pointer to the next element
- Virtual page numbers are compared in this chain searching for a match
  - If a match is found, the corresponding physical frame is extracted





# Hashed Page Table

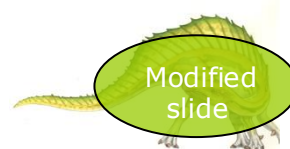




# Swapping

- A process can be **swapped** temporarily out of memory to a backing store, and then brought **back** into memory for continued execution
  - Total memory space of processes can exceed physical memory
- **Backing store** – fast disk large enough to accommodate copies of all memory images for all users; must provide direct access to these memory images
- Major part of swap time is transfer time; total transfer time is directly proportional to the amount of memory swapped
- System maintains a **ready queue** of ready-to-run processes which have memory images on disk

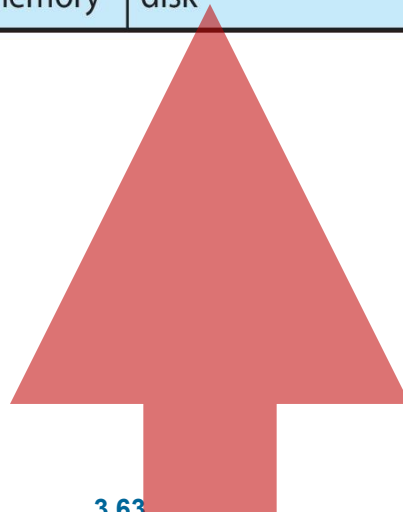
| Level                     | 1                                      | 2                             | 3                | 4                | 5                |
|---------------------------|----------------------------------------|-------------------------------|------------------|------------------|------------------|
| Name                      | registers                              | cache                         | main memory      | solid-state disk | magnetic disk    |
| Typical size              | < 1 KB                                 | < 16MB                        | < 64GB           | < 1 TB           | < 10 TB          |
| Implementation technology | custom memory with multiple ports CMOS | on-chip or off-chip CMOS SRAM | CMOS SRAM        | flash memory     | magnetic disk    |
| Access time (ns)          | 0.25-0.5                               | 0.5-25                        | 80-250           | 25,000-50,000    | 5,000,000        |
| Bandwidth (MB/sec)        | 20,000-100,000                         | 5,000-10,000                  | 1,000-5,000      | 500              | 20-150           |
| Managed by                | compiler                               | hardware                      | operating system | operating system | operating system |
| Backed by                 | cache                                  | main memory                   | disk             | disk             | disk or tape     |





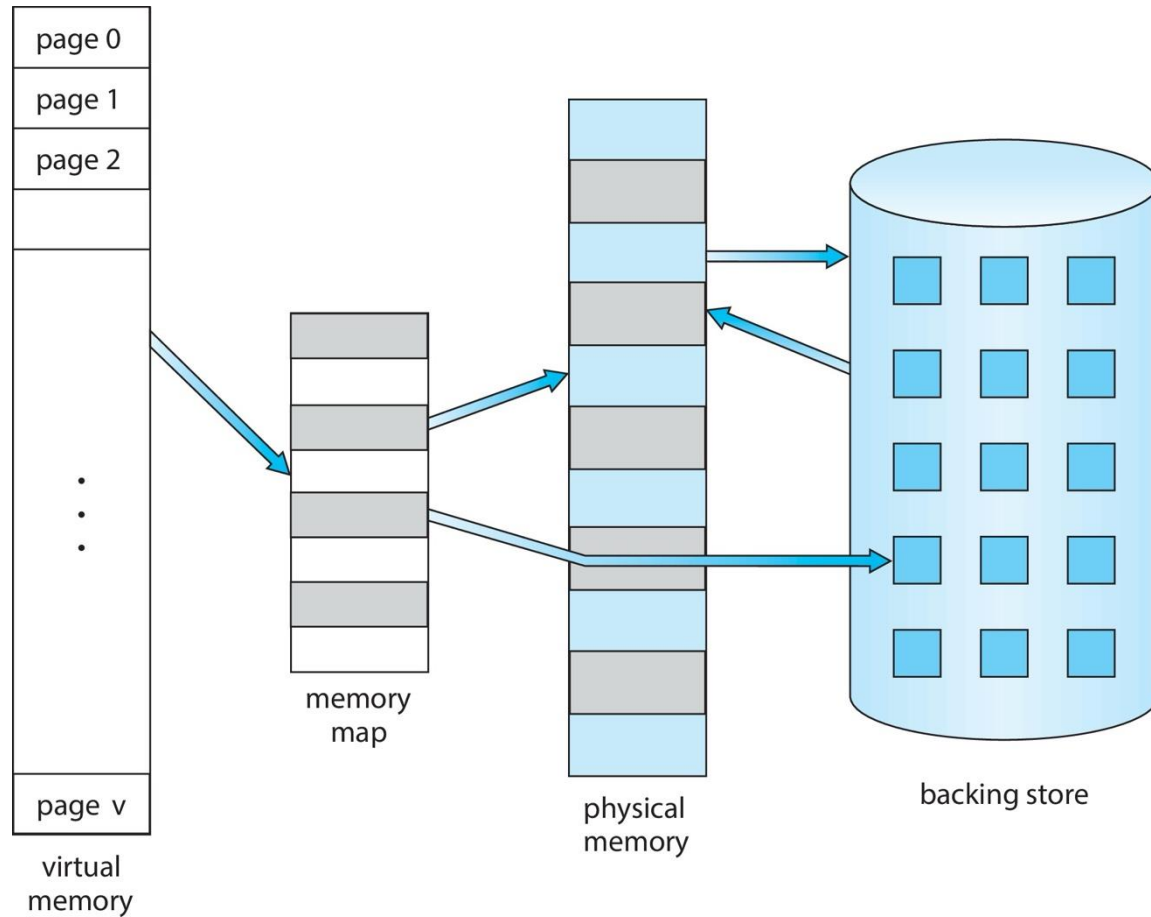
# Characteristics of Various Types of Storage

| Level                     | 1                                      | 2                             | 3                | 4                | 5                |
|---------------------------|----------------------------------------|-------------------------------|------------------|------------------|------------------|
| Name                      | registers                              | cache                         | main memory      | solid-state disk | magnetic disk    |
| Typical size              | < 1 KB                                 | < 16MB                        | < 64GB           | < 1 TB           | < 10 TB          |
| Implementation technology | custom memory with multiple ports CMOS | on-chip or off-chip CMOS SRAM | CMOS SRAM        | flash memory     | magnetic disk    |
| Access time (ns)          | 0.25-0.5                               | 0.5-25                        | 80-250           | 25,000-50,000    | 5,000,000        |
| Bandwidth (MB/sec)        | 20,000-100,000                         | 5,000-10,000                  | 1,000-5,000      | 500              | 20-150           |
| Managed by                | compiler                               | hardware                      | operating system | operating system | operating system |
| Backed by                 | cache                                  | main memory                   | disk             | disk             | disk or tape     |





# Virtual Memory That is Larger Than Physical Memory





# Swapping (Cont.)

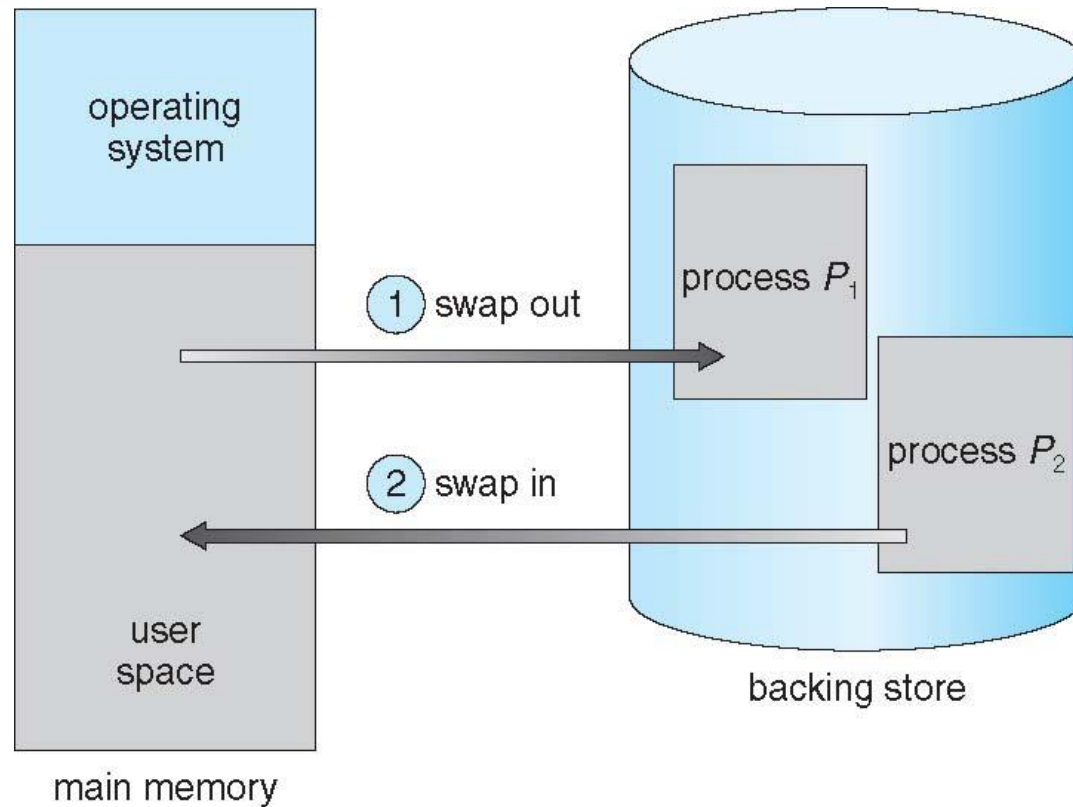
---

- Does the swapped out process need to swap back in to same physical addresses?
- Depends on address binding method
  - Plus consider pending I/O to / from process memory space
- Modified versions of swapping are found on many systems (i.e., UNIX, Linux, and Windows)
  - Swapping normally disabled
  - Started if more than threshold amount of memory allocated
  - Disabled again once memory demand reduced below threshold





# Schematic View of Swapping



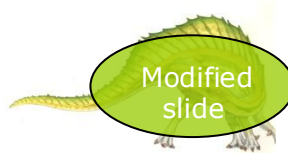




# Context Switch Time including Swapping

---

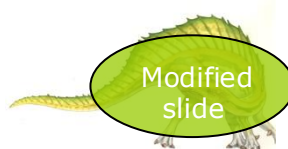
- If next processes to be put on CPU is not in memory, need to swap out a process and swap in target process
- Context switch time can then be very high





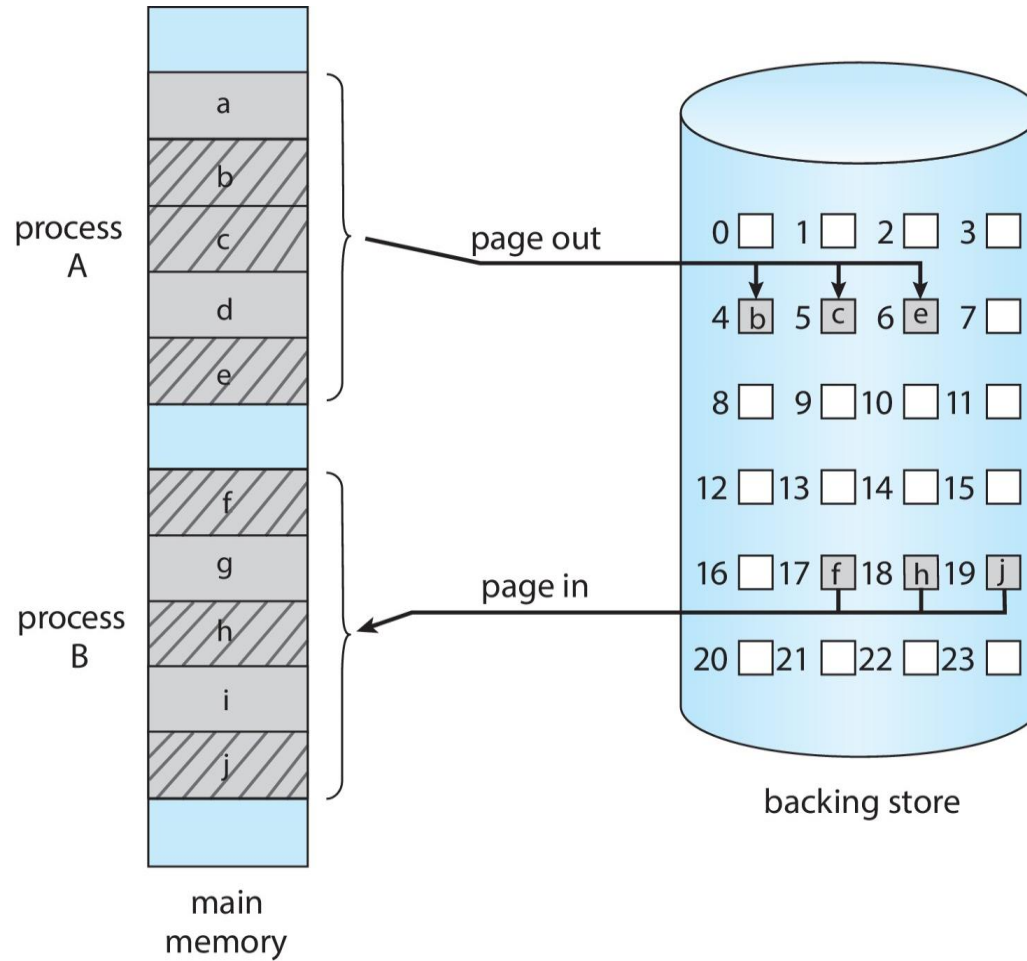
# Context Switch Time and Swapping (Cont.)

- Other constraints as well on swapping
  - Pending I/O – can't swap out as I/O would occur to wrong process
  - Or always transfer I/O to kernel space, then to I/O device
- *Naive* swapping not used in modern operating systems
  - But modified version common
    - ▶ Swap only when free memory extremely low
- **Thrashing**: where our virtual memory is overused, and we need to keep transferring back and forth between our main memory and a backing store. “Page fault”





# Swapping with Paging





# Page Table When Some Pages Are Not in Main Memory

|   |   |
|---|---|
| 0 | A |
| 1 | B |
| 2 | C |
| 3 | D |
| 4 | E |
| 5 | F |
| 6 | G |
| 7 | H |

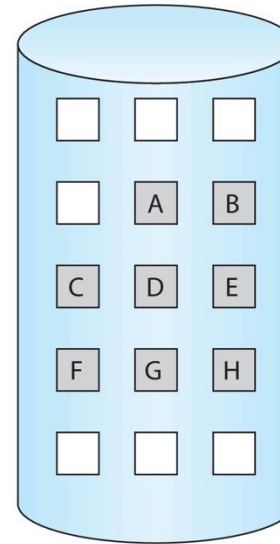
logical  
memory

| valid-invalid<br>bit |   |   |
|----------------------|---|---|
| frame                |   |   |
| 0                    | 4 | v |
| 1                    |   | i |
| 2                    | 6 | v |
| 3                    |   | i |
| 4                    |   | i |
| 5                    | 9 | v |
| 6                    |   | i |
| 7                    |   | i |

page table

|    |   |
|----|---|
| 0  |   |
| 1  |   |
| 2  |   |
| 3  |   |
| 4  | A |
| 5  |   |
| 6  | C |
| 7  |   |
| 8  |   |
| 9  | F |
| 10 |   |
| 11 |   |
| 12 |   |
| 13 |   |
| 14 |   |
| 15 |   |

physical memory



backing store

